

通用演出级卡牌构筑肉鸽引擎

Web / Three.js 技术选型方案

版本 v1.0 - TypeScript-first Web-native Card Theater Engine

本文档给出一套以 Web 技术栈实现通用卡牌构筑肉鸽引擎的技术方案。它继承此前 UCRE 设计文档中的核心思想：规则内核独立、表现层事件驱动、内容数据编译、随机可复现、Replay 可验证。

核心建议：用 TypeScript 编写纯规则内核，用 Three.js 构建桌面剧场，用 React/Vite 构建游戏 UI、编辑器、调试器和回放工具，用 Electron 发行桌面版。

项目	内容
项目代号	UCRE - Universal Card Roguelike Engine
技术路线	Web-first / TypeScript-first / Three.js 桌面剧场
目标玩法	Black Jacket-like、Slay the Spire-like、Inscription-like，以及更多卡牌构筑肉鸽变体
文档用途	技术选型、架构边界、实现路线、风险控制

目录

1. 总体结论
2. 为什么 Web/Three.js 适合本项目
3. 推荐技术栈总表
4. 总体架构
5. 规则内核技术方案
6. Three.js 桌面剧场技术方案
7. 演出系统技术方案
8. 内容与 DSL 管线
9. 存档、Replay 与本地数据库
10. 测试、模拟与 CI
11. 桌面与 Web 发行
12. 三个压力测试用例的落地方式
13. Web 方案风险与规避
14. 实施路线图
15. 参考资料

1. 总体结论

在“不需要开放世界、不需要角色在空间中自由行走、不需要复杂物理，只需要桌面、敌人、卡牌、筹码、棋盘槽位、镜头和演出”的约束下，Web/Three.js 是可行且更适合 AI 辅助开发的主方案。

推荐将项目定义为一个 Web-native Card Theater Engine：规则层负责产生可复现的事实，Three.js 表现层负责把事实转化为桌面剧场演出，React 工具层负责 UI、编辑器、调试器、Replay 和内容生产。

```
主选型：
TypeScript + React + Vite
Three.js WebGL2 桌面剧场
GSAP Timeline + 自研 Presentation Director
纯 TypeScript 规则内核
Zod 内容校验
Dexie / IndexedDB 本地存档
Vitest + Playwright 测试
Electron 桌面发行
```

2. 为什么 Web/Three.js 适合本项目

该项目的核心复杂度主要集中在规则、触发、区域、胜负、构筑、奖励、肉鸽生命周期、编辑器和调试工具，而不是大型场景、NavMesh、角色控制器或复杂物理。因此，不应把 Unity/Unreal 的整套大型游戏引擎能力作为默认前提。

- AI 对 TypeScript、React、YAML、测试代码、工具界面和 Web 调试流程的辅助效率通常更高。
- Three.js 足够表达桌面、卡牌、敌人立牌、筹码、天平、牌堆、槽位、粒子和镜头。
- React/Vite 可以同时承担游戏 UI、内容编辑器、Replay Viewer、规则调试器和平衡面板。
- 规则内核可以在浏览器、Web Worker、Node CLI、Electron 和测试环境中复用。
- Web DevTools、热更新、测试框架和 CI 流程更适合快速迭代复杂卡牌内容。

应避免把 Web 方案做成“浏览器里的 Unity”。更合适的方向是做一个只服务卡牌桌面剧场的窄域引擎。

3. 推荐技术栈总表

层级	推荐选型	说明
主语言	TypeScript	规则、UI、编辑器、模拟器统一语言，利于 AI 生成和维护。
应用框架	React + Vite	React 负责 UI/编辑器；Vite 负责快速开发和构建。
3D 渲染	Three.js WebGL2	当前主线，兼容性和稳定性最好。
高端渲染备选	Three.js WebGPU	作为 feature flag，暂不作为默认渲染路径。
Three 集成	React Three Fiber	可用于场景声明，但高频动画应走 imperative Three refs。
演出时间轴	GSAP Timeline + 自研 Beat Scheduler	负责暂停、跳过、快进、嵌套时间轴和多对象同步。
规则内核	纯 TypeScript 包	不依赖 DOM、React、Three、GSAP，可在 Worker/Node 运行。
内容校验	Zod + JSON Schema	同一套 schema 同时提供运行时校验和 TS 类

层级	推荐选型	说明
		型推断。
本地存档	IndexedDB + Dexie	适合浏览器、PWA、Electron 的离线优先数据。
测试	Vitest + Playwright	规则单测、浏览器测试、E2E、Replay 回归。
桌面发行	Electron	锁定 Chromium，图形行为更可控，适合 Steam/itch。
Web 发行	PWA / 静态站点	适合 Demo、编辑器、Mod 工具和共享预览。

4. 总体架构

```
ucore-web/  
├── apps/  
│   ├── game/           # 正式游戏客户端，React + Three.js  
│   ├── editor/         # 内容编辑器  
│   ├── replay-viewer/  # 回放查看器  
│   └── sandbox/        # 演出和机制实验场  
├── packages/  
│   ├── core/           # 纯规则内核  
│   ├── rulesets/       # blackjack / energy-combat / sacrifice-board  
│   ├── content-schema/ # Zod schema、内容类型、校验器  
│   ├── content-compiler/ # YAML/JSON5 -> runtime bundle  
│   ├── simulation/     # 自动模拟、bot、平衡测试  
│   ├── replay/         # command log / event log / state hash  
│   ├── presentation-core/ # PresentationIntent、Beat、Timeline 数据结构  
│   ├── theater-three/  # Three.js CardActor、ZoneAnchor、VFX、Camera  
│   ├── ui/             # React UI 组件库  
│   └── devtools/       # Rule Debugger、Inspector、Event Viewer  
├── content/  
└── assets/
```

总体架构的关键是三层隔离：规则层只处理事实，表现层只消费事件和表现意图，UI/编辑器只提交命令和展示状态。

层	职责	禁止事项
规则层	GameState、Zone、Command、Effect、Event、Trigger、Objective、RNG、Replay	禁止依赖 DOM、React、Three、GSAP、Date.now、Math.random、requestAnimationFrame
表现层	CardActor、ZoneAnchor、Camera、VFX、Audio、PresentationDirector	禁止直接改规则状态；只能消费 RuleEvent 和 PresentationIntent
UI/工具层	命令构建、状态展示、编辑器、调试器、Replay Viewer	禁止绕过规则内核直接修改 GameState

5. 规则内核技术方案

规则内核使用纯 TypeScript 包实现，命名为 @ucore/core。它应能在浏览器主线程、Web Worker、Node CLI、Electron、测试环境中运行。

```
export interface RuleEngine {
  getState(): Readonly<GameState>;
  dispatch(command: Command): RuleResult;
  replay(commands: Command[], seed: string): ReplayResult;
  serialize(): SerializedGameState;
  getLegalCommands(actorId: ActorId): LegalCommand[];
}

export interface RuleResult {
  accepted: boolean;
  errors: RuleError[];
  events: RuleEvent[];
  presentationIntents: PresentationIntent[];
  stateHash: string;
}
```

规则内核必须保持确定性。所有随机必须来自显式 RNG Stream；所有排序必须稳定；所有状态变化必须产生事件；所有事件必须可序列化。

- Object + Zone：卡牌、敌人、遗物、状态、槽位、天平、筹码池都是对象或区域。
- Command Pipeline：玩家/AI/剧情的所有行动统一走命令管线。
- Effect Resolver：只有 Effect 能修改 GameState。
- Event Ledger：所有规则事实 append-only 记录。
- Trigger Queue：触发、替代、防止、持续效果按时机窗口结算。
- Objective Graph：胜负条件通过配置和谓词表达，不写死为 HP <= 0。
- Replay Runner：seed + command log + content manifest hash 必须可复现同一 event log。

6. Three.js 桌面剧场技术方案

Three.js 只承担桌面剧场，不承担规则真源。桌面剧场包含桌面、牌堆、手牌、敌人、棋盘槽位、筹码池、遗物、天平、粒子、灯光和镜头。

```
Scene
├── CameraRig
├── Lights
├── TableRoot
│   ├── TableMesh
│   ├── BoardSlots
│   ├── BlackjackSlots
│   ├── ScaleObject
│   └── Pot / Chips
├── CardLayer
│   ├── CardActorPool
│   ├── HandCards
│   ├── BoardCards
│   └── PileCards
├── EnemyLayer
│   ├── EnemyBust
│   ├── IntentObjects
│   └── BossVFX
└── VFXLayer
```

每张卡牌是一个 CardActor。规则层通过 objectId 引用卡牌，表现层通过 ActorRegistry 把 objectId 映射到 Three.js 对象。

```
export interface CardActor {
```

```
objectId: string;
group: THREE.Group;
cardMesh: THREE.Mesh;
frontMesh: THREE.Mesh;
backMesh: THREE.Mesh;
borderMesh?: THREE.Mesh;
material: CardMaterialHandle;
currentAnchor?: ZoneAnchorId;
visualState: CardVisualState;
}
```

卡面建议用 Canvas2D/SVG 生成纹理，小卡使用 Three texture，悬停审视和奖励选择使用 React DOM 高清卡面，以保证文字清晰和本地化便利。

7. 演出系统技术方案

演出系统由 PresentationIntent、PresentationDirector、BeatScheduler、GSAP Timeline、ActorRegistry、AnchorRegistry、CameraDirector、VFXDirector 和 AudioDirector 组成。

```
Rule Event:
  CardMoved(card_103, player.hand -> player.discardPile)

PresentationIntent:
  type: CardMove
  subject: card_103
  fromAnchor: player.hand
  toAnchor: player.discardPile
  style: paper-slide

PresentationDirector:
  resolve actor
  resolve anchors
  build GSAP timeline
  play movement / flip / sound / VFX
  update actor currentAnchor
```

GSAP 解决“怎么动”，PresentationDirector 解决“为什么动、何时动、哪些事件合并成一个演出段落”。

演出类型	规则事件	Three/GSAP 表现
抽牌	CardMoved(drawPile -> hand)	卡牌从牌堆飞入手牌扇形，轻微翻转和纸牌声。
打牌	CardPlayed + CardMoved	卡牌飞向目标或桌面槽位，镜头轻推，冲击 VFX。
销毁	ObjectDestroyed	shader burn/dissolve、灰烬粒子、卡牌回收对象池。
献祭	ObjectSacrificed + ObjectDestroyed	卡面渗血、血球飞向费用计、暗色音效。
Bust	ObjectiveCompleted / BustCheck	卡面烧红、点数爆裂、筹码散开、镜头震动。
遗物触发	TriggerResolved	遗物图标闪光、粒子飞向相关对象。

8. 内容与 DSL 管线

内容源推荐使用 YAML / JSON5 / JSON，经过内容编译器校验和规范化后输出 runtime content bundle。设计师和 AI 都能稳定编辑声明式内容，而不需要进入 Unity/UE 的 Inspector、Prefab、Scene 和 Timeline 系统。

```
id: card.blood_wager
name: Blood Wager
types: [Skill, Gamble]
cost:
  coins: 2
timing:
  playableDuring: PlayerDecision
effects:
  - GainResource:
      resource: bet
      amount: 4
  - DrawCards:
      amount: 1
presentation:
  profile: casino_chip_burst
```

内容编译流程：

```
YAML / JSON5 source
→ Zod schema validation
→ content lint
→ rule validation
→ presentation binding validation
→ canonical JSON bundle
→ manifest hash
→ runtime load
```

建议前期先提供 30-50 个内置 Effect，覆盖 MoveObject、DrawCards、DealDamage、GainResource、SacrificeObject、DestroyObject、CompareHandTotal、TipScale 等核心动作，避免过早设计任意脚本语言。

9. 存档、Replay 与本地数据库

本地存档使用 IndexedDB + Dexie。Replay 不存每一帧画面，只存内容版本、初始 seed、ruleset、command log、关键 snapshot 和校验 hash。

```
ReplayData {
  replayId: string;
  contentManifestHash: string;
  rulesVersion: string;
  seed: string;
  rulesetId: string;
  commandLog: Command[];
  snapshots?: StateSnapshot[];
  finalStateHash: string;
  finalEventHash: string;
}
```

为了保证长期可复现，任何存档或 Replay 都必须记录 schemaVersion、rulesVersion、contentManifestHash、enabledMods、seed 和 commandLog。

10. 测试、模拟与 CI

Web/TypeScript 方案的优势之一是测试成本低。规则测试用 Vitest，浏览器交互测试用 Vitest Browser Mode 或 Playwright，完整流程用 Playwright E2E，平衡模拟用 Node CLI 或 worker_threads。

测试类型	推荐工具	目标
规则单测	Vitest	Effect、Trigger、Objective、Zone 移动、资源变化。
Golden Replay	Vitest + ReplayRunner	固定 seed 和 command log 得到固定 state hash。
浏览器测试	Vitest Browser Mode	Canvas 启动、卡牌 Hover、目标选择、简单演出。
端到端测试	Playwright	从开始 Run 到战斗、奖励、地图推进。
平衡模拟	Node CLI	自动跑 1000-10000 局，输出胜率、卡牌选择率、资源曲线。
性能测试	浏览器 Performance + 自定义计数器	FPS、draw call、对象池、粒子数量、GC。

```
CI Pipeline:
Pull Request
→ TypeScript typecheck
→ ESLint / Prettier
→ Content Lint
→ Rule Unit Tests
→ Golden Replay Tests
→ Simulation Smoke Test
→ Browser Smoke Test
→ Electron Build Smoke
```

11. 桌面与 Web 发行

正式 PC 版建议优先 Electron，因为 Electron 自带 Chromium，渲染行为更可控，WebGL2/WebGPU 能力更可预测，DevTools 和 Steam/itch 发行流程更成熟。

发行方式	适合场景	备注
浏览器 / PWA	Demo、编辑器、内容预览、Mod 工具	天然 Web 入口，方便分享和测试。
Electron	Steam/itch 桌面正式版	锁定 Chromium，渲染一致性强。
Tauri	轻量工具、低包体积发行	依赖系统 WebView，图形能力差异更大。
移动/PWA	后期探索	首发不建议以移动为主目标。

若未来需要 PvP、排行榜或云同步，规则层已经天然适合迁移到 Node.js 服务端权威模拟，但早期不应过度投入后端。

12. 三个压力测试用例的落地方式

Web/Three.js 方案要通过 Black Jacket-like、Slay the Spire-like、Inscription-like 三种极端用例验证通用性。

用例	Three.js 表现对象	规则模块
Black Jacket-like	牌桌、牌靴、玩家/敌人手牌区、Sleeve 区、筹码池、Winner Pot、Boss Curse 悬浮物	HandValueEvaluator、BettingService、PotService、SleeveZone、HollowStack、WhisperTrigger、DrainValueEffect、BossCurseRuleObject
Slay-like	抽牌堆、手牌扇形、弃牌堆、Exhaust 区、敌人立牌、Intent 图标、遗物栏、药水栏	Energy、DrawDiscardCycle、Block、Damage、Intent、RelicTrigger、PowerLayer、RewardDraft、MapGraph
Inscription-like	木桌、双方四格槽位、敌人队列、天平、蜡烛、Side Deck、物品架、Sigil 图标	LaneBoard、Slot、SacrificeCost、BonesOnDeath、SideDeck、SigilKeyword、ScaleObjective、LaneAttackResolver

13. Web 方案风险与规避

风险	严重度	规避方案
WebGPU 尚未完全统一	高	WebGL2 作为主线；WebGPU 只做可选高级画质开关。
React 每帧重渲染导致卡顿	高	React 管 UI；Three Mesh 用 refs；GSAP/自研 tween 直接改 transform。
卡牌文字在 3D 中不清晰	中高	小卡使用 CanvasTexture；悬停审视使用 DOM 高清卡面。
复杂规则阻塞主线程	中高	规则引擎迁移到 Web Worker；自动模拟走 Node worker_threads。
存档和 Replay 版本混乱	高	强制记录 contentManifestHash、rulesVersion、schemaVersion、seed、commandLog。
粒子和透明材质性能不稳	中	VFX LOD、粒子池、DPR 限制、低配模式。
内容 DSL 过早失控	中高	先做声明式 effect 组合，避免任意脚本。

14. 实施路线图

阶段	主要内容	验收标准
阶段 1: 规则核心纵切	GameState、Object、Zone、MoveObject、DrawCards、PlayCard、Discard、Destroy、Resource、EventLog、TriggerQueue、ObjectiveCheck、SeededRNG	Vitest 单测通过；Golden Replay 可复现。
阶段 2: Three 桌面纵切	桌面、手牌扇形、抽牌堆、弃牌堆、CardActor、ZoneAnchor、基础飞牌	卡牌能从牌堆飞入手牌，从手牌飞入弃牌堆。
阶段 3: 演出导演	PresentationIntent、BeatScheduler、GSAP Timeline、CameraDirector、CardMoveProfile、DestroyProfile、Skip/FastForward	跳过/加速演出不改变规则状态。
阶段 4: 三玩法压力测试	Slay-like、Blackjack-like、Inscription-like 最小规则集	三个玩法都能在同一 core 上运行。
阶段 5: 编辑器与 Replay	Card Editor、Ruleset Editor、Replay Viewer、Rule Debugger、Content Lint	设计师/AI 可通过 YAML/JSON 创建内容并验证。
阶段 6: 正式垂直切片	完整 Run 生命周期、商店、事件、奖励、Boss、演出主题、Electron 打包	可交付一条可玩的高表现 demo。

15. 参考资料

- Vite Guide: <https://vite.dev/guide/>
- Three.js WebGLRenderer Documentation: <https://threejs.org/docs/pages/WebGLRenderer.html>
- MDN WebGPU API: https://developer.mozilla.org/en-US/docs/Web/API/WebGPU_API
- Three.js WebGPU Renderer Manual: <https://threejs.org/manual/en/webgpurenderer>
- React Three Fiber Introduction: <https://r3f.docs.pmnd.rs/getting-started/introduction>
- GSAP Timeline Documentation: <https://gsap.com/docs/v3/GSAP/Timeline/>
- Zod Documentation: <https://zod.dev/>
- Dexie Documentation: <https://dexie.org/>
- Vitest Browser Mode: <https://main.vitest.dev/guide/browser/>
- Node.js Release Schedule: <https://github.com/nodejs/release>
- Electron Releases: <https://releases.electronjs.org/>
- Tauri Start Guide: <https://tauri.app/start/>